

CSE 6220 / CX 4220 – Introduction to High-Performance Computing

HW2 Solutions

Problem 1

At the beginning of the algorithm, A stores only the initial values for the first row and first column of the tableau. Subsequently, as the `tableau` function computes the i^{th} row, the new values for $A(i, 1), A(i, 2), \dots, A(i, N-1)$ get stored in $A[i+(N-2)], A[i+(N-3)], \dots, A[i]$, leaving the value of $A[i-1] = A(i-1, N-1)$ (i.e., last column from the previous row) intact. The new value for $A(i, j)$ replaces the old value of $A(i-1, j-1)$, but that's ok, since the algorithm no longer needs the value of $A(i-1, j-1)$ after $A(i, j)$ has been computed. At the end of the computation, A stores the values for the last column and last row.

Problem 2

1. $\Theta(n^2/B)$
2. $\Theta(n/B)$

Problem 3

The work is $\Theta(n^2)$ while the span is $\Theta(n^{\lg_k 2k-1})$.

Problem 4

1. $Q(n) = \Theta(n/B)$ if $n < cM$, otherwise $Q(n) = 4Q(n/2) + \Theta(1)$.
2. The height is $\lg(n/cM)$.
3. The number of leaves is n^2/c^2M^2 .
4. $\Theta(n^2/MB)$.

Problem 5

If we are unlucky, we would be forced to recursively sub-divide until the subpieces are small enough to fit into the cache, which can be as small as $n < cM/k$. This results in the recurrence relation

$$Q(n) = \Theta(n/B) \text{ if } n < cM/k, \text{ otherwise } Q(n) = k^2Q(n/k) + \Theta(1)$$

which gives us $Q(n) = \Theta(n^2k/MB)$.

On the other hand, if we are lucky, the recurrence relation would be

$$Q(n) = \Theta(n/B) \text{ if } n < cM, \text{ otherwise } Q(n) = k^2Q(n/k) + \Theta(1)$$

which gives us $Q(n) = \Theta(n^2/MB)$.